

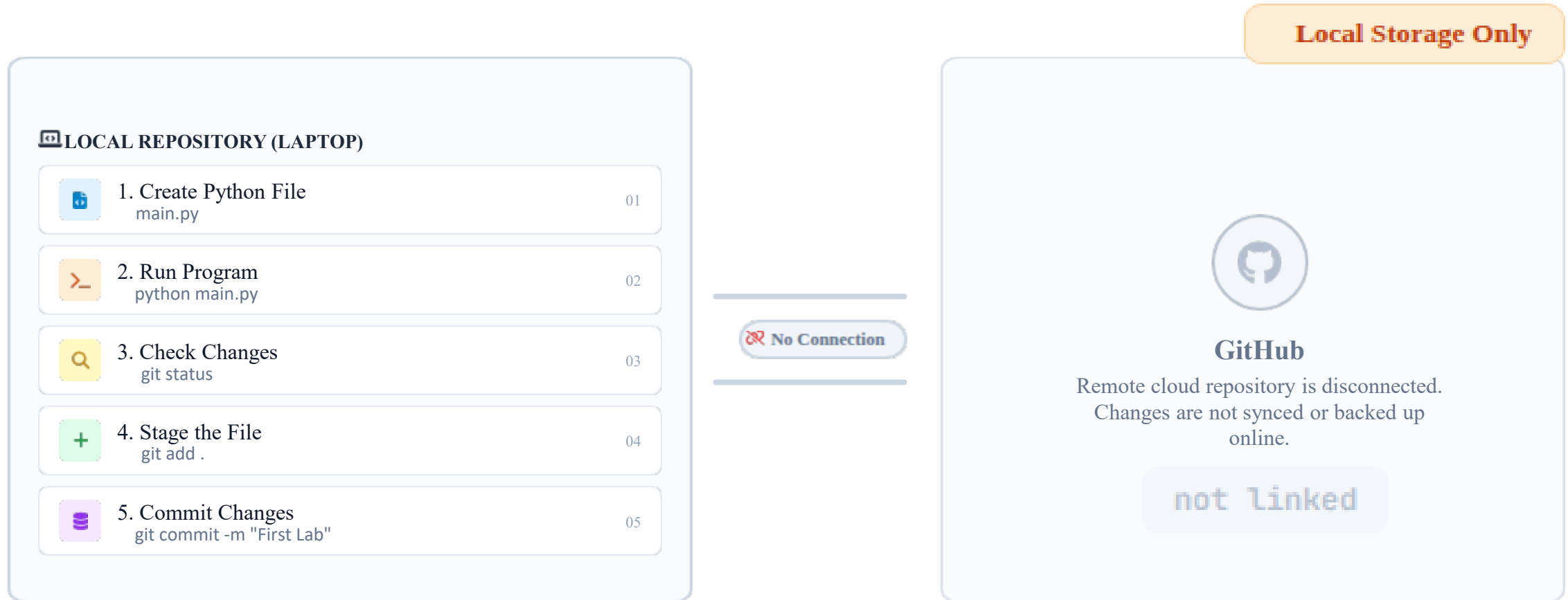


INF1103: Programming Fundamentals with DevOps

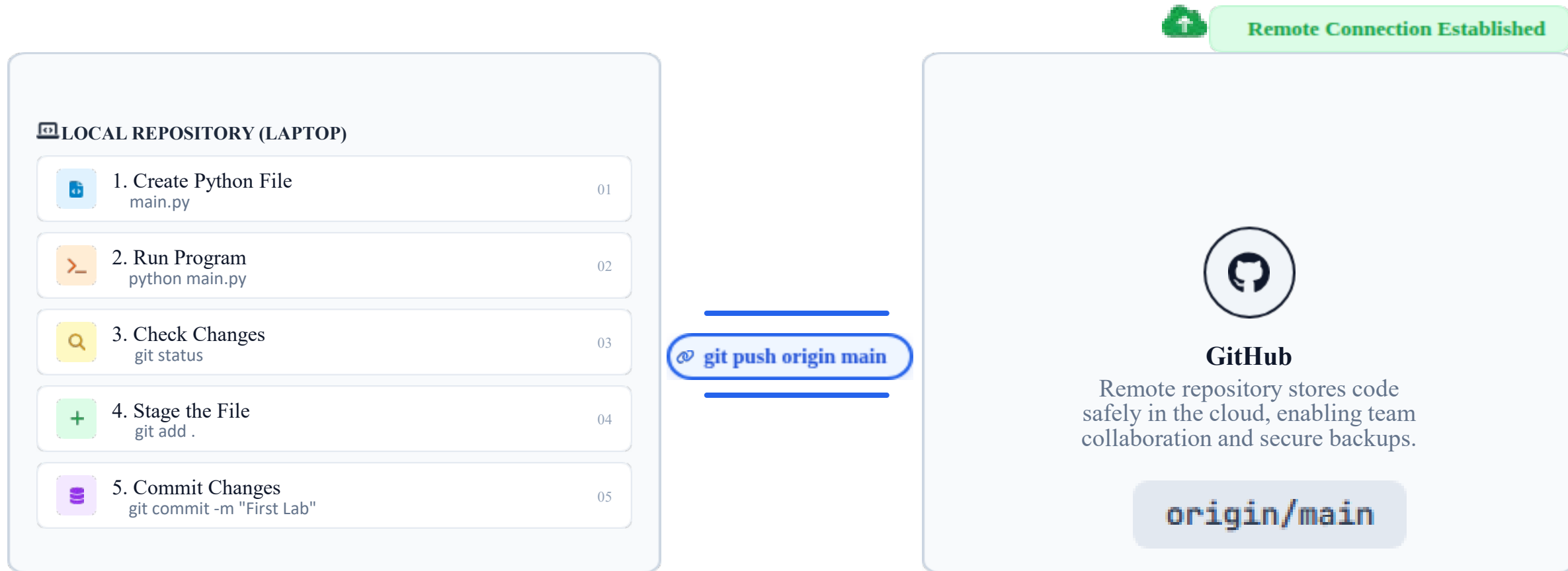
Week 1: DevOps & Docker: Standardizing the Environment from Day
One

Python Local Repository

Without remote setup, your work remains entirely isolated on your local laptop hardware.



Python Local Repository to GitHub Connect



GitHub Workflow

Fetching and merging remote updates from GitHub into your local workspace.

LOCAL REPOSITORY (LAPTOP)

Receives incoming changes, automatically merges remote modifications, and updates your local working directory.



Merge & Resolve

Update local python scripts

 `git pull origin main`



GitHub Remote

Contains team edits, peer reviews, or cloud commits made by collaborators.

Source:

Remote Origin

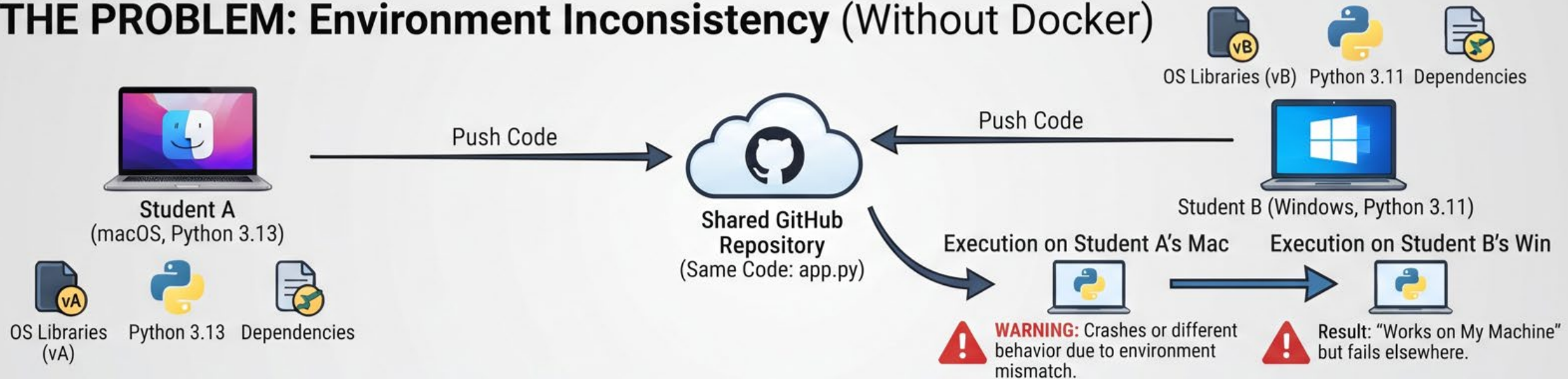
Action:

Fetch + Merge

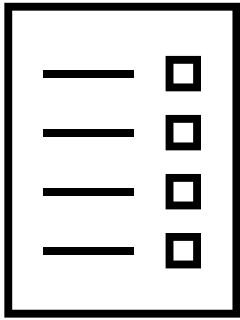
Download Updates

Inconsistency Issue

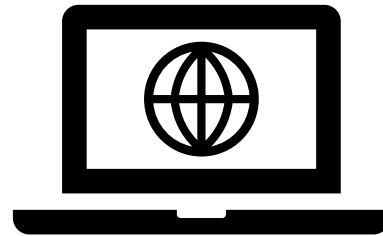
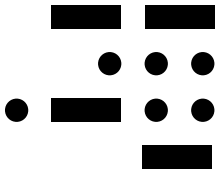
THE PROBLEM: Environment Inconsistency (Without Docker)



Environmental Drift



Missing libraries or
mismatched versions
cause 90% of
deployment failures.



Apple iOS



The “Works on my Machine” Problem

- Baking a perfect chocolate cake at home.
- Write the recipe down to the exact gram
- Mail it to a friend living high in the mountains.
- But the cake collapses in the oven





What is DevOps

Development (The Chefs)

The engineering team **writing the software recipes**. They are motivated by speed, rapid innovation, continuous updates, and deploying features as fast as possible to meet user demands.

Operations (The Kitchen Staff)

The systems team running the **physical kitchen**. They are motivated by stability, system uptime, security, and reducing risk. Naturally, rapid changes from the developers threaten this stability.

DevOps bridges this divide. It is a culture where developers build applications with operations, deployment, and hosting in mind from Day One.

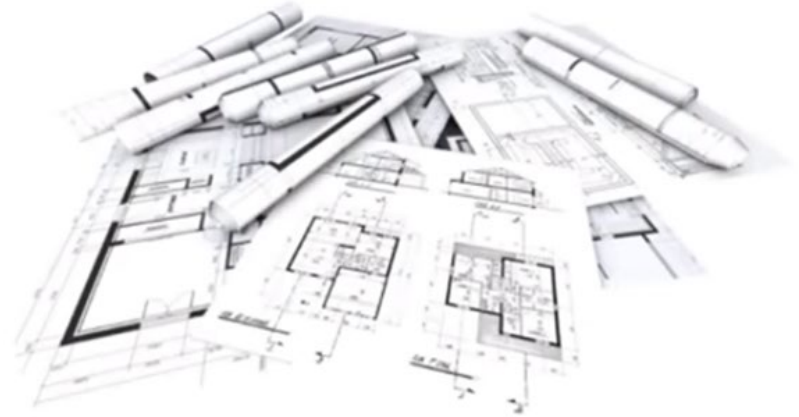


Docker packages code with its exact environment, making it portable and reproducible.

Docker Pillar 1: The Image

The Immutable Blueprint

- A Docker Image is a read-only, frozen snapshot of an environment. It packages your code, runtime, operating system libraries, and system configurations into a single file.
- Because it is immutable (unchangeable), once an image is compiled, it guarantees 100% repeatable execution across any host machine.



The Global Shipping Matrix

- **Prior to the 1950s**
 - Cargo was manually packed. Wood barrels, sacks of flour, and pianos were stuffed together on a ship. It was chaotic, fragile, and required full manual repackaging at every harbor-to-train exchange.
- **The Container Solution**
 - Introduce the standard, intermodal box. Transport infrastructure stopped worrying about the cargo inside and focused purely on moving one predictable shape.



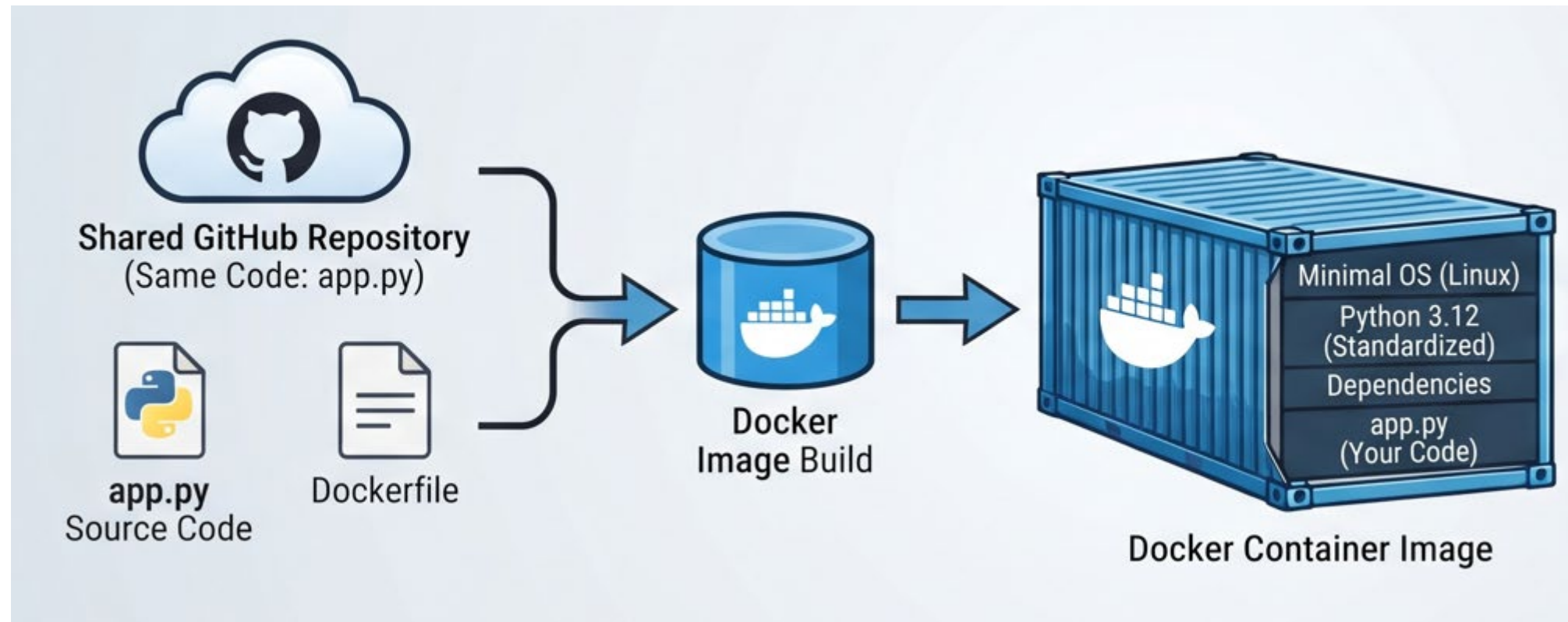
Docker Pillar 2: The Container

The Active Sandbox

- A Docker Container is the live, running instance built directly from the Image blueprint. It is isolated, lightweight, and completely detached from other containers and your laptop's global background filesystem.
- Containers share your computer's resources dynamically, unlike heavy and slow Virtual Machines (VMs).



Standardization Solution



Supporting Infrastructure



The Dockerfile

The Recipe Book

A simple, plain-text instruction manual where you declare your system specifications (e.g., base image, dependencies to install, files to copy, and startup commands).



The Registry

The Public Library

A central cloud warehouse (like Docker Hub) where pre-built images are shared. You can search for and instantly pull images like Python, Postgres, or Node.js.



The Volume

The External Drive

Since containers are ephemeral (disposable), deleting a container deletes its data. Volumes mount folders from your laptop inside the container to save database records permanently.



Why We Docker

 **Clean Laptops:** No global databases or runtimes cluttering your computer state.

 **Instant Resets:** If database configs break, delete the container and spin a fresh one up in 2 seconds.

 **Team Harmony:** Shared Dockerfiles ensure "it works on my machine" conflicts are completely eliminated.

GitHub vs. Docker

GitHub (The Written Record)

- Manages Text and History
- **What it stores:** Source code files, markdown instructions, and the revision history of who changed what, when, and why.
- **Layman Analogy:** A collaborative architect's sketchbook. It organizes, displays, and versions the floorplans. It cannot build the house; it only holds the plans.

Docker (The Physical Builder)

- Manages Environments and Runtimes
- **What it packages:** Operating system kernels, system dependencies, port bindings, and active execution runtimes
- **Layman Analogy:** The construction crew. They read the blueprint, pack all required resources into structured shipping containers, and build the physical house identically anywhere.

The Collaborative Tag-Team Pipeline



1. Design & Commit

You write your Python logic and a custom Dockerfile blueprint.

You push this source code repository up to GitHub to sync with your team.

2. Build & Test

An automated cloud runner picks up the code from GitHub, uses Docker to compile it into an image, and runs isolated verification test suites.

3. Run & Deploy

The validated Docker image is deployed to live production servers, spinning up seamlessly and matching your local laptop configuration exactly.

Takeaway!!

- Docker containers ensure consistent application environments, solving deployment and configuration problems.
- Can learn command-line skills, pulling images, executing containers, and observing application outputs.
- Standardizing improves team collaboration and consistency in software development projects.

Thanks!!!

